

space for a package. The name space management specifies which variables are imported from other packages, and describes the variables that are exported to users of the package.

```
$ cat NAMESPACE
## Re-written 2012-11-03 B. D. Ripley
useDynLib(bitops, .registration = TRUE, .fixes = "C_")
export(bitFlip, bitAnd, bitOr, bitXor, bitShiftL, bitShiftR,
cksum)
export("%&%", "%|%", "%^%", "%<<%", "%>>%" )
```

- **man:** This directory contains manual pages for the source package, and is written in the 'R documentation' (Rd) format.
- **R:** This directory contains the actual R source files of the project.
- **src:** This directory contains additional source files that are used by the package, such as .c and .h header files that interact with R's foreign function interface (FFI).

```
$ ls src/
bit-ops.c bit-ops.h cksum.c init.c
```

- **tests:** This folder has the complete test suite to run and verify the R package. For example:

```
$ ls tests/
consistency.R
```

- **DESCRIPTION:** This file provides the meta-information with details of the package name, version, date, author, maintainer, licence, URL, repository, published date, etc.

Build

You can build a package from the R source directory files using the *R CMD build* command. The *bitops.1.0.7* sources can be built from the command line, as shown below:

```
$ R CMD build bitops
* checking for file 'bitops/DESCRIPTION' ... OK
* preparing 'bitops':
* checking DESCRIPTION meta-information ... OK
* cleaning src
* checking whether 'INDEX' is up-to-date ... NO
* use '--force' to remove the existing 'INDEX'
* checking for LF line-endings in source and make files and
shell scripts
* checking for empty or unneeded directories
* building 'bitops_1.0-7.tar.gz'
```

You can also manually install a package from the shell command line after downloading the tarball with the *R CMD*

INSTALL command, as shown below:

```
$ R CMD INSTALL bitops_1.0-7.tar.gz
* installing to library '/home/shakthi/R/x86_64-pc-linux-gnu-
library/4.1'
* installing *source* package 'bitops' ...
** package 'bitops' successfully unpacked and MD5 sums
checked
** using staged installation
** libs
gcc -I"/usr/include/R/" -DNDEBUG -D_FORTIFY_SOURCE=2
-fpic -march=x86-64 -mtune=generic -O2 -pipe -fno-plt -c
bit-ops.c -o bit-ops.o
gcc -I"/usr/include/R/" -DNDEBUG -D_FORTIFY_SOURCE=2
-fpic -march=x86-64 -mtune=generic -O2 -pipe -fno-plt -c
cksum.c -o cksum.o
gcc -I"/usr/include/R/" -DNDEBUG -D_FORTIFY_SOURCE=2
-fpic -march=x86-64 -mtune=generic -O2 -pipe -fno-plt -c
init.c -o init.o
gcc -shared -L/usr/lib64/R/lib -Wl,-O1,--sort-common,--as-
needed,-z,relro,-z,now -o bitops.so bit-ops.o cksum.o init.o
-L/usr/lib64/R/lib -lR
installing to /home/shakthi/R/x86_64-pc-linux-gnu-
library/4.1/00LOCK-bitops/00new/bitops/libs
** R
** byte-compile and prepare package for lazy loading
** help
*** installing help indices
** building package indices
** testing if installed package can be loaded from temporary
location
** checking absolute paths in shared objects and dynamic
libraries
** testing if installed package can be loaded from final
location
** testing if installed package keeps a record of temporary
installation path
* DONE (bitops)
```

You can verify the same using the *packageVersion* command in the R console:

```
> packageVersion("bitops")
[1] '1.0.7'
```

The *devtools* library provides various functions that allow you to install R packages from Git repositories and URLs. This is useful if you would like to test the latest main development branch of an R package. You will need to first load the *devtools* library in the R console session:

```
> install.packages("devtools")
```